

Entropiq — Технические требования

Персональный финансовый помощник с учётом инфляции и системой прогнозирования финансовых целей

Версия документа: 1.0

Дата: 2026-02-08

Статус: Утверждён

1. Общее описание

Веб-сервис для учёта личных и семейных финансов с функцией пересчёта покупательной способности с учётом инфляции и AI-рекомендациями для достижения финансовых целей. Freemium-модель монетизации.

Целевая аудитория: резиденты РФ (молодые профессионалы 25–35 лет, семьи с детьми 30–45 лет).

Рабочее название: Entropiq

Расположение проекта: `~/domains/hse/entropiq`

Репозиторий: GitHub (приватный)

2. Архитектура

2.1. Тип архитектуры

Монолитное приложение с модульной внутренней структурой (Service classes, Action classes, Events/Listeners). Модульность позволяет при необходимости выделить отдельные сервисы в будущем.

2.2. Основные модули

Модуль	Описание
Auth	Регистрация, аутентификация (SMS), авторизация
Transactions	Учёт доходов и расходов, категории, повторяющиеся операции
Goals	Финансовые цели, прогресс, прогнозирование
Inflation	Интеграция с ЕМИСС, расчёт ИПЦ, пересчёт покупательной способности
AI	Анализ паттернов, генерация рекомендаций, сценарное моделирование
Reports	Аналитика, графики, экспорт PDF/Excel
Subscriptions	Управление подписками, платежи, freemium-логика
Notifications	Email, SMS, Telegram, push-уведомления
Users	Профили, настройки, мультиязычность, будущие семейные аккаунты

2.3. API

- REST API с версионированием: `/api/v1/...`
- API проектируется с первого дня с расчётом на будущее мобильное приложение
- Документация API: автоматическая генерация (Scribe или Scramble)

3. Технологический стек

3.1. Backend

Компонент	Технология	Версия
Язык	PHP	8.4
Фреймворк	Laravel	12.x
Аутентификация	Laravel Sanctum	—
Очереди	Laravel Queues (Redis driver)	—
Мониторинг очередей	Laravel Horizon	—
Планировщик задач	Laravel Scheduler	—
Логирование действий	Spatie Activity Log	—
API-интеграции	Saloon	—
Экспорт Excel	Laravel Excel (Maatwebsite)	—
Экспорт PDF	DomPDF / Browsershot	—
Тестирование	Pest	—
Браузерное тестирование	Laravel Dusk	—
Статический анализ	Larastan (PHPStan)	Level 6+
Код-стайл	Laravel Pint	—

3.2. Frontend

Компонент	Технология
CSS-фреймворк	Tailwind CSS
Реактивность (server-side)	Livewire + Livewire Volt
UI-компоненты	Flux
Реактивность (client-side)	Alpine.js
Графики	ApexCharts

3.3. Базы данных и хранилища

Компонент	Технология	Назначение
Основная БД	PostgreSQL 16	Данные приложения
Кэш, сессии, очереди	Redis 7	Кэширование, session store, queue broker
Полнотекстовый поиск	PostgreSQL FTS	Поиск по операциям

3.4. Мультивалютность

Структура БД проектируется с поддержкой нескольких валют с первого дня. Все денежные суммы хранятся в минорных единицах (копейки/центы) как `bigint`. Каждая транзакция содержит поле `currency_code` (ISO 4217). Базовая валюта — RUB.

3.5. Мультиязычность

Интернационализация через Laravel-хелпер `__()` с первого дня. Языковые файлы в `lang/`. Стартовые языки: ru, en.

4. Внешние интеграции

Все интеграции реализуются через пакет **Saloon** (единообразные коннекторы, retry-логика, кэширование, логирование).

Интеграция	Провайдер	Назначение
LLM (primary)	GigaChat / YandexGPT	AI-советы, анализ паттернов
LLM (fallback)	OpenAI API	Резервный AI-провайдер
SMS	SMS Aero	Аутентификация по SMS, уведомления
Данные инфляции	API ЕМИСС (Росстат)	Индекс потребительских цен
Платёжная система	TBD (ЮKassa — приоритетный кандидат)	Приём платежей за подписку
Email-маркетинг	Unisender	Рассылки, email-цепочки
Уведомления	Telegram Bot API	Push-уведомления в Telegram

5. Аутентификация и авторизация

5.1. Аутентификация

- **Метод:** номер телефона + одноразовый SMS-код (OTP)
- **Провайдер SMS:** SMS Aero
- **Срок жизни кода:** 5 минут
- **Rate limiting:** не более 3 попыток ввода кода, не более 1 SMS в 60 секунд на номер
- **Будущее расширение:** OAuth (VK, Google), email + пароль

5.2. Авторизация

- **Пакет:** Laravel Sanctum
- **Веб-интерфейс:** cookie-based сессии (SPA stateful auth)
- **API (мобильное приложение):** Bearer-токены
- **Управление доступом:** Laravel Policies + Gates

- **Роли:** реализуются при необходимости (admin, user, family_member — на втором этапе)

6. Инфраструктура

6.1. Контейнеризация

Окружение	Инструмент	Описание
Local (разработка)	Laravel Sail	PHP 8.4, PostgreSQL 16, Redis 7, Mailpit
Stage (тестирование)	docker- compose.stage.yml	PHP-FPM + Nginx, оптимизированные образы, без Xdebug
Production	docker- compose.prod.yml	PHP-FPM + Nginx, без dev-зависимостей, healthchecks

6.2. Хостинг

Окружение	Платформа	Конфигурация
Stage	Timeweb VPS	Один VPS, Docker Compose на выделенных портах
Production	Timeweb VPS (отдельный)	Поднимается позже, выделенный сервер

6.3. Docker-сервисы

Local (Sail):

- app (PHP 8.3 + extensions)
- postgresql
- redis
- mailpit
- meilisearch (опционально)

Stage/Production:

- app (PHP 8.3-FPM + Nginx)
- postgresql
- redis
- horizon (worker для очередей)
- scheduler (cron)

6.4. SSL

Let's Encrypt (Certbot) для stage и production.

7. CI/CD

7.1. CI — GitHub Actions

Пайплайн запускается на каждый PR:

1. **Lint:** Laravel Pint (проверка код-стайла)
2. **Static Analysis:** Larastan (PHPStan level 6+)
3. **Tests:** Pest (unit + feature)
4. **Build:** проверка сборки Docker-образа

7.2. CD — Laravel Envoy

- **Stage:** автоматический деплой при мерже в ветку `stage`
- **Production:** ручной запуск деплоя при мерже в ветку `main`
- **Процесс деплоя:** `pull` → `composer install (--no-dev)` → `npm build` → `migrate` → `cache clear/warm` → `restart Horizon workers` → `healthcheck`

7.3. Git-стратегия ветвления

feature/* → PR → stage → PR → main

↓

Stage-сервер

↓

Production-сервер

hotfix/* → PR → main (+ обратный мерж в stage)

- **main** — production-ready код
- **stage** — интеграционная ветка, деплоится на stage-сервер
- **feature/*** — функциональные ветки, создаются от stage
- **hotfix/*** — экстренные исправления, создаются от main

8. Мониторинг и отладка

Инструмент	Окружение	Назначение
Laravel Telescope	Local, Stage	Отладка запросов, jobs, exceptions, queries
Laravel Pulse	Stage, Production	Мониторинг приложения (performance, slow queries, queues)
Sentry	Production	Error tracking, alerting
UptimeRobot	Production	Мониторинг доступности (внешний пинг)
Netdata	Production	Мониторинг сервера (CPU, RAM, disk, network)

Telescope отключён в production для экономии ресурсов.

9. Безопасность

- HTTPS обязателен во всех окружениях (кроме local)
- CSRF-защита для всех форм (Laravel middleware)
- Rate limiting на API-эндпоинты (Laravel Rate Limiter)

- Валидация входных данных через Form Requests
- Хранение секретов через `.env` (не коммитятся в репозиторий)
- Параметризованные запросы (Eloquent ORM — SQL-инъекции исключены)
- Content Security Policy headers
- Audit trail через Spatie Activity Log (логирование всех критичных операций)
- Соответствие 152-ФЗ (обработка персональных данных)

10. Бэкапы

- **Метод:** самописный bash-скрипт (`pg_dump` → сжатие → выгрузка на внешнее хранилище)
- **Частота:** каждые 6 часов (cron)
- **Хранение:** внешнее хранилище (S3-совместимое или отдельный сервер)
- **Ретеншн:** 7 дней (28 бэкапов)
- **Будущее:** миграция на Spatie Laravel Backup

11. Тестирование

Тип	Инструмент	Покрытие
Unit-тесты	Pest	Бизнес-логика, расчёты инфляции, модели
Feature-тесты	Pest	API-эндпоинты, аутентификация, авторизация
Browser-тесты	Laravel Dusk	Критические пользовательские сценарии
Email-тестирование	Mailpit	Проверка шаблонов и доставки (local)

12. Этапность реализации

Этап 1 (MVP) — февраль–июнь 2026

- Аутентификация по SMS
- CRUD операций (доходы/расходы)
- Категории транзакций
- Финансовые цели с учётом инфляции
- Интеграция с ЕМИСС (данные ИПЦ)
- Базовые AI-советы (LLM)
- Дашборд с графиками
- Freemium-модель (ограничения бесплатной версии)
- REST API v1
- Подписки и платежи
- Email-уведомления
- Экспорт PDF/Excel
- Мультиязычность (ru, en)
- Мультивалютная структура БД (активная валюта — RUB)

Этап 2 (после защиты) — июль–декабрь 2026

- Семейные аккаунты (мультиюзер на один бюджет)
 - Service Worker / PWA (оффлайн-добавление операций)
 - Реферальная программа
 - Продвинутые AI-советы (ML, анализ сезонности)
 - Мобильное приложение (PWA или натив — решение позже)
 - OAuth-аутентификация (VK, Google)
 - Интеграция с банками (автоимпорт операций)
 - Telegram-бот
 - Партнёрские программы
-

13. Структура проекта (ориентировочная)

```
entropiq/  
├─ app/  
|   ├─ Actions/           # Single-action классы  
|   ├─ Http/  
|   |   ├─ Controllers/  
|   |   |   └─ Api/  
|   |   |       └─ V1/   # Версионированные API-контроллеры  
|   |   └─ Middleware/  
|   |   └─ Requests/     # Form Requests (валидация)  
|   └─ Models/  
|   └─ Policies/  
|   └─ Services/         # Бизнес-логика  
|   └─ Integrations/     # Saloon коннекторы  
|   |   └─ Emiss/  
|   |   └─ GigaChat/  
|   |   └─ SmsAero/  
|   |   └─ ...  
|   └─ Jobs/  
|   └─ Events/  
|   └─ Listeners/  
|   └─ Notifications/  
|   └─ Enums/  
├─ config/  
├─ database/  
|   └─ migrations/  
|   └─ seeders/  
|   └─ factories/  
├─ lang/  
|   └─ ru/  
|   └─ en/  
├─ resources/  
|   └─ views/  
|       └─ components/  
|       └─ livewire/
```

```
|       └─ layouts/
├─ routes/
|   ├─ web.php
|   ├─ api.php
|   └─ console.php
├─ tests/
|   ├─ Unit/
|   ├─ Feature/
|   └─ Browser/
├─ docker/                # Docker-конфиги для stage/prod
|   ├─ nginx/
|   └─ php/
├─ docker-compose.yml     # Sail (local)
├─ docker-compose.stage.yml
├─ docker-compose.prod.yml
├─ Envoy.blade.php        # Деплой-скрипты
├─ .github/
|   └─ workflows/
|       └─ ci.yml         # GitHub Actions
├─ pint.json
├─ phpstan.neon
└─ .env.example
```

14. Открытые вопросы

#	Вопрос	Статус	Дедлайн
1	Выбор доменного имени	Открыт	До настройки stage
2	Выбор платёжной системы (ЮKassa или альтернатива)	Требуется ресёрч	До этапа подписок
3	Мобильное приложение: PWA vs натив vs кроссплатформа	Открыт	Этап 2
4	Выбор LLM-провайдера (GigaChat vs YandexGPT)	Требуется сравнение API	До реализации AI-модуля